# FACTORYMIND CDT

**PROJECT DOCUMENTATION — INITIAL SUBMISSION**

An Uncertainty-Aware Cognitive Digital Twin for Evidence-Grounded Manufacturing Diagnosis, Simulation, and Decision Support

| Project Name | FactoryMind CDT |
|---|---|
| Track | Machine Learning Engineer |
| Team Leader | محمد عادل رمضان عبد الحميد |
| Team Size | 6 students |
| Required Sections | Project Planning & Management; Literature Review; Requirements Gathering |

# Team Roster and Responsibility Allocation

The names below follow the Arabic team roster provided. Role allocation is the team's proposed workstream split and should be confirmed by all members.

| Student ID | Official Name (Arabic) | Assigned Workstream |
|---|---|---|
| 21003072 | محمد عادل رمضان عبد الحميد | Team Lead; System Architecture; API Contracts & Integration; Integration QA |
| 21140977 | محمد أسامة عبد المحسن عبد العزيز | Data Engineering; Dataset Audit; Data Dictionary; Validation & Preprocessing |
| 21046871 | بيشوي سعد بشاره مخائيل | Time-Series ML; Anomaly Detection; Forecasting; Model Evaluation |
| 21010230 | أحمد عصام راشد عبد العال | NLP; Technical Documentation; Retrieval & Reranking |
| 21029403 | علي معتصم ممدوح عبد | Evidence Graph; Root Cause Analysis (RCA); Uncertainty & Abstention |
| 21128637 | زياد أحمد ربيع علي | Bounded Digital Twin; Impact Calculator; Demo UI |

# 1. Project Planning & Management

## 1.1 Project Proposal and Overview

Problem: Manufacturing teams may investigate equipment anomalies using fragmented sensor readings, maintenance records, quality reports, and operating context. An isolated alert may lack traceable evidence.

Proposed solution: A decision-support prototype combining authorized manufacturing data, ML anomaly analysis, evidence retrieval, explicit uncertainty, and one bounded scenario simulation. It provides candidate explanations for human review and does not control machinery.

Target users: Maintenance engineers, production engineers, quality engineers, and supervisors.

## 1.2 Objectives

- Prepare a reproducible pipeline for one selected dataset/simulator.
- Build and evaluate a baseline anomaly-detection or failure-classification method, depending on labels.
- Link incident results to evidence and label candidate causes as hypotheses.
- Demonstrate one bounded what-if scenario with stated assumptions.
- Measure model quality, false alarms, evidence traceability, and response time.

## 1.3 Scope and Boundaries

Included: one use case/dataset, data cleaning, baseline ML, incident-review interface, evidence links, one bounded scenario, evaluation, documentation. Excluded: direct machine/PLC control, automatic setpoint changes, unsupported proprietary integrations, and untested real-factory claims.

## 1.4 Proposed Work Plan and Milestones

Proposed internal plan based on team-stated 6 December 2026 final deadline. The guideline lists 16 October for initial DEPI sections; the announcement supplied by the team says 19 October. Confirm applicable deadline with instructor.

| Period | Milestone | Deliverable |
|---|---|---|
| 19–25 Oct | Scope and data | Use case, dataset, data dictionary, backlog, acceptance criteria |
| 26 Oct–1 Nov | Data and baseline | Preprocessing, split strategy, baseline metric, limitations |
| 2–8 Nov | ML prototype | Model, evaluation, error review |
| 9–15 Nov | Evidence investigation | Incident view, source records, uncertainty |
| 16–22 Nov | Simulation and UI | Bounded scenario and dashboard |
| 23–29 Nov | Integration and tests | Integrated prototype and test results |

| | | |
|---|---|---|
| 30 Nov–4 Dec | Final evaluation | Results, limitations, guide, report, slides |
| 5–6 Dec | Final handoff | Repository, README, demo, submission |

## 1.5 Task Assignment & Roles

Task allocation follows the team's proposed six-workstream division. The team should approve these assignments and designate a backup reviewer for each workstream.

| Team Member | Assigned Responsibility |
|---|---|
| محمد عادل رمضان عبد الحميد | Team Lead; System Architecture; API Contracts & Integration; Integration QA |
| محمد أسامة عبد المحسن عبد العزيز | Data Engineering; Dataset Audit; Data Dictionary; Validation & Preprocessing |
| بيشوي سعد بشاره مخائيل | Time-Series ML; Anomaly Detection; Forecasting; Model Evaluation |
| أحمد عصام راشد عبد العال | NLP; Technical Documentation; Retrieval & Reranking |
| علي معتصم ممدوح عبد | Evidence Graph; Root Cause Analysis (RCA); Uncertainty & Abstention |
| زياد أحمد ربيع علي | Bounded Digital Twin; Impact Calculator; Demo UI |

## 1.6 Resources and Dependencies

- Python analysis/ML tools and GitHub; choose dashboard framework after scope/data confirmation.
- Public or authorized dataset/simulator; check license, provenance, labels, units, missing values.
- Six team members, instructor, and domain reviewer if available.
- Dependencies: data access, stable environment, agreed metrics, feedback.

## 1.7 Risk Assessment & Mitigation

| Risk | Impact | Mitigation |
|---|---|---|
| No labels | High | Choose early; use anomaly detection if needed |
| Data leakage | High | Appropriate split; fit preprocessing on training only |
| False alarms | High | Report metrics and errors; human review |
| Broad scope | High | One bounded scenario; state assumptions |
| Integration delay | Medium | Define interfaces; integrate weekly |
| Privacy/licensing | High | Authorized data; record license |
| Schedule slippage | High | MVP, milestones, escalate blockers |

## 1.8 KPIs

| KPI | Measurement | Provisional acceptance |
|---|---|---|
| Model quality | Task-appropriate metrics | Set after data/baseline inspection |
| False alarms | False positives/reviewed sample | Report rate and examples |
| Evidence traceability | Evidence items with source IDs | 100% source identifiers |
| Response time | Incident selection to results | Set after profiling |
| Reproducibility | Second member runs README | One clean successful run |
| Testing | Critical tests passed | All critical workflow tests pass |

## 1.9 Workstream Dependencies and Integration

The six workstreams will be integrated through agreed data schemas and API contracts. Work can proceed in parallel once the shared contracts are agreed; the sequence below describes the main dependencies.

- Data Engineering (محمد أسامة عبد المحسن عبد العزيز) defines the dataset schema, data dictionary, validation checks, and reproducible prepared-data output.
- Time-Series ML (بيشوي سعد بشاره مخائيل) consumes validated data and returns anomaly/forecast outputs with timestamps, model version, and evaluation results.

- NLP and Retrieval (أحمد عصام راشد عبد العال) prepares technical-document chunks and returns ranked evidence with source identifiers and retrieval scores.
- Evidence Graph and RCA (علي معتصم ممدوح عبد) combines model outputs and retrieved evidence into candidate cause hypotheses, records supporting or conflicting evidence, and represents uncertainty or abstention.
- Digital Twin and UI (زياد أحمد ربيع علي) consumes agreed outputs and runs bounded what-if scenarios; the interface displays assumptions and does not control real equipment.
- Team Lead (محمد عادل رمضان عبد الحميد) defines and reviews shared schemas/API contracts, integrates components, and coordinates integration tests.

*Shared contract fields to agree before implementation: event_id, timestamp, asset_id (if available), data_source_id, model_version, anomaly_score or predicted_label (when supported), evidence_source_id, candidate_hypothesis, uncertainty_status, scenario_id, and evaluation metadata. If a field is unavailable in the selected dataset, mark it unavailable rather than inventing a value.*

# 2. Literature Review

## 2.1 Review Focus and Method

Focused initial review of manufacturing digital-twin frameworks, predictive maintenance, ML diagnosis, and industrial data integration; not exhaustive.

## 2.2 Similar Work and Comparison

| Source | Contribution | Limitations / relevance |
|---|---|---|
| ISO 23247-1:2021 | Manufacturing digital-twin framework | Framework, not ready-made diagnosis model |
| Shao & Helu (2020) | Manufacturing digital-twin framework | Does not prove model accuracy in a specific plant |
| Carvalho et al. (2019) | Review of ML for predictive maintenance | Results depend on data, labels, and context |
| Dalzochio et al. (2020) | ML and reasoning for predictive maintenance | Deployment and integration context-dependent |
| NIST ISO 23247 use cases | Implementation scenarios | Do not replace project-specific validation |

## 2.3 Research Gap and Differentiation

Proposed differentiator: connect anomaly to inspectable evidence, show candidate causes with uncertainty, and compare one bounded scenario. This remains a design objective until evaluated.

## 2.4 Feedback, Evaluation, Suggested Improvements

Lecturer feedback: pending; no assessment supplied. Improvements: expand search after choosing exact use case, add domain-specific sources, compare baseline with stronger model, conduct error analysis, assess uncertainty if supported, gather user feedback if available, revise after instructor review.

## 2.5 Final Grading Criteria

Guidelines mention documentation, implementation, testing, and presentation but no numeric weights. Request official rubric. Internal checks: completeness, correctness, evaluation, reproducibility, limitations, team contributions, demo quality.

## 2.6 References


- ISO (2021). ISO 23247-1:2021, Digital twin framework for manufacturing, Part 1. https://www.iso.org/standard/75066.html
- Shao & Helu (2020). Framework for a digital twin in manufacturing. Verify exact bibliographic details.
- Carvalho et al. (2019). A systematic literature review of ML methods applied to predictive maintenance. Computers & Industrial Engineering, 137, 106024. https://doi.org/10.1016/j.cie.2019.106024
- Dalzochio et al. (2020). Machine learning and reasoning for predictive maintenance in Industry 4.0. Computers in Industry, 123, 103298. https://doi.org/10.1016/j.compind.2020.103298
- NIST. Use case scenarios for digital twin implementation based on ISO 23247. Verify exact metadata.


Verify exact metadata for Shao & Helu and NIST entries against original records before submission.

# 3. Requirements Gathering

## 3.1 Stakeholder Analysis

| Stakeholder | Needs | Validation |
|---|---|---|
| Maintenance engineer | Inspect alerts, evidence, likely causes | Incident walkthrough |
| Production supervisor | Understand impact, compare safe options | Review scenario assumptions |
| Quality engineer | Relate anomalies to quality events | Review evidence links |
| Project team / ML engineer | Reliable pipeline, metrics, reproducibility | Code/metric review |
| Instructor/evaluator | Scope, justified methods, reproducible demo | Milestone/final review |

## 3.2 User Stories and Use Cases

| ID | User story | Acceptance condition |
|---|---|---|
| US-01 | Select event and inspect signals/records | Event and records shown; missing data flagged |
| US-02 | View model output to prioritize review | Model/version and score/class shown |
| US-03 | See candidate causes linked to evidence | Source IDs shown; hypotheses separated from facts |
| US-04 | Compare bounded scenario | Baseline/scenario/assumptions shown; no machine changes |
| US-05 | Reproduce pipeline from README | Another member runs demo |

## 3.3 Functional Requirements

| ID | Requirement | Priority |
|---|---|---|
| FR-01 | Ingest authorized dataset/simulator export | Must |
| FR-02 | Validate fields, types, ranges, missing values | Must |
| FR-03 | Record source and preprocessing | Must |
| FR-04 | Split train/validation/test appropriately | Must |
| FR-05 | Baseline anomaly detection/classification per labels | Must |
| FR-06 | Report metrics and confusion matrix when possible | Must |
| FR-07 | Select event for investigation | Must |
| FR-08 | Display supporting signals/records | Must |
| FR-09 | Label explanations as hypotheses | Must |
| FR-10 | Show uncertainty when supported; disclose limitations | Should |
| FR-11 | Provide bounded scenario comparison | Should |
| FR-12 | Record model/data versions and evaluation | Should |
| FR-13 | Show missing-data/processing errors | Must |
| FR-14 | Provide setup/execution README | Must |
| FR-15 | Never send commands to production equipment | Must |
| FR-16 | Reviewer notes if in scope | Could |

## 3.4 Non-functional Requirements

| ID | Quality | Requirement |
|---|---|---|
| NFR-01 | Performance | Measure render/process time; set target after profiling |
| NFR-02 | Reliability | Repeatable runs where method permits |
| NFR-03 | Security | Authorized data; no secrets in repository |
| NFR-04 | Privacy | No confidential data without authorization/protection |
| NFR-05 | Usability | Reviewer can select event and inspect evidence |

| | | |
|---|---|---|
| NFR-06 | Traceability | Evidence source IDs and model/data versions recorded |
| NFR-07 | Maintainability | Separate data, model, evaluation, UI modules |
| NFR-08 | Portability | Dependencies and tested setup instructions |
| NFR-09 | Safety | Read-only; no autonomous control |
| NFR-10 | Transparency | Document assumptions, limitations, failure cases |

## 3.5 Data Requirements and Constraints

- Potential fields: timestamps, equipment/process IDs, sensor values, labels if available, maintenance logs, quality records.
- Record source, license, units, missingness, label meaning, and limitations.
- Prevent leakage from future/post-failure information unavailable at prediction time.
- Do not assume live factory connectivity or proprietary APIs without permission.

## 3.6 Acceptance and Change Control

Review requirements at milestones; accept only with recorded test/demo evidence. Track scope/data/model changes in GitHub issues or project tracker and confirm material changes with instructor.